Partially Recurrent Network With Highway Connections

Jianyong Wang, *Member, IEEE*, Haixian Zhang, *Member, IEEE*, Zhang Yi, *Fellow, IEEE*

***Abstract*—It is difficult to train deep recurrent neural networks (RNNs) to learn the complex dependencies in sequential data, because of gradient problems and learning conflicts. To address these learning problems, this paper proposes a partially recurrent network (PR-Net), which consists of the partially recurrent (PR) layers with highway connections. The proposed PR layer explicitly arranges the neurons into a memory module and an output module, which are implemented by highway networks. The memory module places memory on the information highway, to maintain long short-term memory. The output module places external input on the information highway, to approximate the complex transition function from one step to the next. Because of the highway connections in the output module, the PR layer can pass gradient information across layers easily, to allow the training of the PR-Net with multiple stacked PR layers. Furthermore, the explicit separation of the memory module and output module can reduce the learning burden of RNNs that which neurons are responsible for memory and which for output. With a comparable number of parameters, sublayers in the recurrent layer, and stacked recurrent layers, the proposed PR-Net outperformed the recurrent highway network and long short-term memory on three sequence learning benchmarks, significantly.**

***Index Terms*—Recurrent Neural Network; Long Short-Term Memory; Highway Network; Sequence Learning**

I. Introduction

Recently, deep neural network (DNN) has achieved impressive results in various applications, such as computer vision [1]–[6] and natural language processing [7], [8]. As a class of DNN, recurrent neural networks (RNNs) are widely used in the field of sequence learning [7]–[10]. It is a class of biologically inspired artificial neural networks that model the feedback connections by recurrent connections [11]. Because of the recurrent connections, RNNs can maintain the working memory (activations of neurons), and merge the memory of the history with the current information to produce predictions [12].

For RNNs, the main obstacles to learning long-term dependencies are learning conflicts and gradient problems. After investigating the input conflict and output conflict in RNNs, Hochreiter and Schmidhuber proposed a gating mechanism [13], in which the "input operation" and "output operation" of memory units are controlled by additional input gating units and output units, respectively. The gating mechanism has been a common component in state-of-the-art RNN models, such as long short-term memory (LSTM) [13], gated recurrent unit (GRU) [14], and recurrent highway network (RHN) [15]. The memory conflict in RNNs was studied in [12], where it was addressed by the separation of memory from the output.

Compared with learning conflicts, gradient problems are always a greater challenge for RNNs. In sequence learning, an RNN is unfolded into a very deep feedforward neural network (FNN) by assuming that weight connections are fixed across time steps. It is obvious that the depth of the unfolded FNN is proportional to the length of the input sequence [8], [15]. It is difficult to train deep neural networks by using the backpropagation algorithm because the gradient information would decrease or explode exponentially while being back-propagated layer by layer. In the original LSTM [13], the linear memory neuron with fixed recurrent weight 1 and the technique of gradient chunking were proposed to address the gradient problem in RNNs. The experiments demonstrated its effectiveness in the learning of long-term dependencies. Furthermore, in the GRU [14], leaky integration was proposed, in which the forget gate neuron and input neuron were replaced by one updating neuron, achieving comparable performance. The aforementioned mechanisms have been abstracted as highway connections, which allow the training of ultra-deep neural networks, including FNNs and RNNs.

To learn the complex space transformation in sequential data, the spatial depth of RNNs is often increased by stacking more recurrent layers or adding multiple nonlinear sublayers to the recurrent layer. Stacking recurrent layers [16], which is analogous to using multiple hidden layers in FNNs, increases the nonlinearity of RNNs and aggravates the gradient problem, at the same time. To train the stacked RNNs, an effective mechanism, similar to the residual connections and highway connections in FNNs, should be developed to pass the gradient information across stacked layers [8], [17].

Adding multiple nonlinear sublayers to the recurrent layer seems to be a more effective method and has been studied widely. Generally, there are two methods. One is representing the transition function of the recurrent layer by deep FNNs; for example, deep transition RNNs and RHNs [15]. In an RHN [15], each recurrent layer contains several highway layers, and the memory information can pass through sublayers and time steps, to maintain long short-term memory. The other method is introducing "mini tick" into the recurrent layer; in this method, several state transitions are performed to produce the final output at each time step. Graves extended this method to adaptive computation time [18] and achieved comparable results for several sequence learning tasks.

This work was supported by the National Major Science and Technology Projects of China under Grant No. 2018AAA0100201 and the National Natural Science Foundation of China under Grants No. 61906127 and No. 61836011.

Jianyong Wang, Haixian Zhang, Zhang Yi are with Machine Intelligence Laboratory, College of Computer Science, Sichuan University, Chengdu 610065, P. R. China. E-mail: zhangyi@scu.edu.cn

In this paper, we investigate current RNN models and propose a partially recurrent network (PR-Net). The main contributions of the paper are as follows:

- We propose a general model of a partially recurrent (PR) layer, which consists of an output module for deep spatial feature learning and a memory module for maintaining long short-term memory.
- The proposed PR layer is implemented with highway connections. The memory module places memory state on the information highway and aims to maintain long short-term memory. The output module places external input on the information highway and aims to approximate the complex transition function from one step to the next.
- The PR-Net is designed by stacking several PR layers. Because of the highway network in the output module, the PR layer can pass the gradient information across layers easily. It allows the training of the deep PR-Net.
- The proposed PR-Net was compared with the state-of-the-art RNN models on three sequence learning benchmarks. The experiments demonstrated the superior performance of PR-Net.

The remainder of this paper is organized as follows. The notation and related work are presented in Section II. The proposed PR-Net is described in Section III. Experiments and conclusions are discussed in Sections IV and V, respectively.

II. Related works

A. Notations

For convenience, the symbols used throughout this paper are presented in Table I.

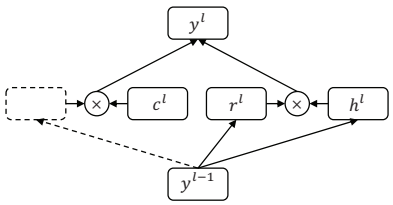
TABLE I
SYMBOLS USED THROUGHOUT THIS PAPER.

Symbol	Description
T	The length of a sequence
t	The index of time step
s_t, y_t	The memory state and forward output at time step t
$\mathcal{R}, \mathcal{F}$	Mappings with and without recurrent connections respectively
L	The number of sub-layers in a recurrent layer
l	The index of sub-layers in a recurrent layer
D	The number of stacked recurrent layers
W	The connection weight
σ	The sigmoid activation function: $\sigma(x) = \frac{1}{1+\exp(-x)}$
$\tanh$	The tangent activation function: $\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$

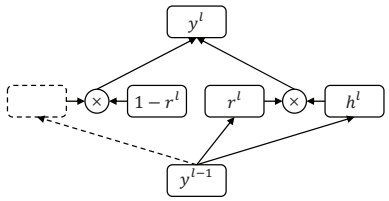
B. Highway Blocks and Highway Connections

The highway block [19] is inspired by LSTM networks. It introduces the highway connection, for information to flow out easily, to allow the training of ultra-deep neural networks with multiple highway blocks. As illustrated in Fig.1, the highway block inherits the highway connection and gating mechanism from LSTM. For convenience, we use a fully connected layer here. The computation of the highway block in Fig.1a can be formulated as:

$$\begin{cases} y^l = y^{l-1} \cdot c^l + r^l \cdot h^l \\ h^l = \tanh(W_h^l y^{l-1}) \\ c^l = \sigma(W_c^l y^{l-1}) \\ r^l = \sigma(W_r^l y^{l-1}), \end{cases} \tag{1}$$



(a)



(b)

Fig. 1. Illustration of highway blocks . The dotted rectangle representes the copy of the input. The symbol '$\times$' denotes the multiplicative operations. h^l, c^l, and r^l are all nonlinear transformations with trainable weights. Furthermore, y^{l-1} and y^l are output of layer $l-1$ and l respectively. (a) A standard highway block with carry gate c^l and transform gate r^l. (b) A simple highway block with the setting $c^l = 1 - r^l$.

where y^{l-1} and y^l are the outputs of layers $l-1$ and l, respectively. h^l is a nonlinear transformation. c^l and r^l are carry gate and transform gate , respectively. It is obvious that the highway block would be generalized to residule block if we set $c^l = 1$ and $r^l = 1$.

For simplicity , the carry gate is always coupled to the transform gate by setting $c^l = 1 - r^l$. This produces the widly used version of highway block:

$$\begin{cases} y^l = y^{l-1} \cdot (\mathbf{1} - r^l) + r^l \cdot h^l \\ h^l = \tanh(W_h^l y^{l-1}) \\ r^l = \sigma(W_r^l y^{l-1}). \end{cases} \tag{2}$$

From Eq. (2), we obtain:

$$\frac{\partial y^l}{\partial y^{l-1}} = \begin{cases} \mathbf{I}, & r^l = \mathbf{0} \\ \frac{\partial h^l}{\partial y^{l-1}}, & r^l = \mathbf{1}. \end{cases} \tag{3}$$

This means that the highway block could behave as a nonlinear transform or a linear layer that simply passes its inputs through [19]. It has been proved that such behaviors eliminate the adverse effect of the gradient problem and allow the training of ultra-deep networks, such as the highway network [19] and RHN [17].

C. Fully Recurrent Layer

The recurrent connection plays a crucial role in RNNs. It allows RNNs to maintain memory and possess rich dynamics. RNNs are extremely powerful in modeling sequences. Given an input sequence $\mathbf{x} = \{x_t \in \mathbb{R}^m | t = 1, 2, \cdots, T\}$, at each time t, a standard recurrent layer updates its memory state $s_t \in \mathbb{R}^n$ and computes the forward output $y_t \in \mathbb{R}^n$ according to

$$\begin{cases} s_t &= \mathcal{R}(s_{t-1}, x_t) \\ y_t &= s_t, \end{cases} \tag{4}$$

where the memory state is initialed as $s_t = \mathbf{0}$. $\mathcal{R}$ is a state transition mapping and could be represented as a neural

network. For example, it is a composition of a hyperbolic tangent function with an affine transformation for simple recurrent network [20], and a highway network for RHN [15]. For LSTM network and AMU [12], $\mathcal{R}$ also contains a recurrent network for memory cell.

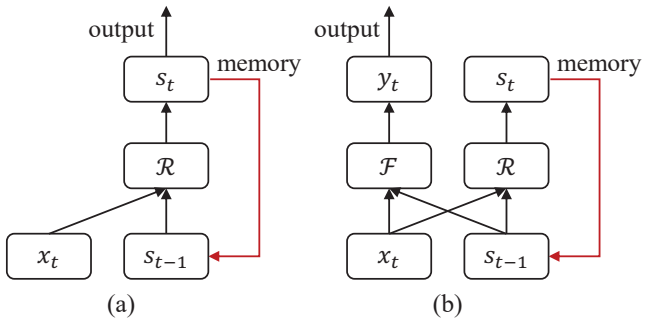


Fig. 2. The architectures of two types of recurrent layers. The red lines represent the recurrent connections. $\mathcal{R}$ and $\mathcal{F}$ denote the memory state transition and forward nonlinear mapping, respectively. (a) Fully recurrent layer, in which all of the outputs feed back to the input through recurrent connections. (b) Partially recurrent layer, in which some of the outputs feed back to the input through the recurrent connections, while the others serve as the forward output of this recurrent layer.

The recurrent layer described in Eq. (4) could be named the fully recurrent (FR) layer. As illustrated in Fig.2, an FR layer has two characteristics. First, all outputs of such recurrent layer feedback through the recurrent connections and serve as input at the next time step. Second, the output and memory share the same neurons. The problem is that an RNN with an FR layer has to learn itself which neurons are responsible for memory and which are responsible for output [18] and it potentially suffers the memory conflict problem [12]. The AMU [12] attempts to address the aforementioned problems by choosing apart of recurrent neurons as output neurons and separating the computation of feedforward nonlinear mapping from that of the recurrent transition. However, the output neurons are still recurrent neurons. To fully utilize the advantage of separating the output neurons and memory neurons, partially recurrent neural network is proposed in this manuscript.

III. Model of Partially Recurrent Neural Network

In this section, the partially recurrent (PR) layer is introduced and effective implementation of the PR layer is presented by using highway connections. The partially recurrent network (PR-Net) consisting of the proposed PR layers is then described and compared with the RHN and LSTM networks in detail. It is proved that the proposed PR-Net can overcome the problems of the fully recurrent layer and improve the performance of RNNs for complex sequence learning.

A. General Formulation of Partially Recurrent Layer

Fig. 3 illustrates the main concepts of the proposed PR layer. In contrast to the FR layer, the PR layer separates the neurons into two modules. The memory module contains a memory state transition $\mathcal{R} : s_{t-1}, x_t \rightarrow s_t$ and the output module contains a forward nonlinear mapping $\mathcal{F} : s_{t-1}, x_t \rightarrow y_t$. The memory state transition $\mathcal{R}$ of the PR layer is similar to that of an FR layer. Generally, $\mathcal{F}$ can be implemented by various networks, such as a standard fully connected layer with a sigmoid activation function, a highway network, etc.

At each time step t, the PR layer updates its memory state and computes the output by:

$$\begin{cases} s_t &= \mathcal{R}(s_{t-1}, x_t) \\ y_t &= \mathcal{F}(s_{t-1}, x_t). \end{cases} \tag{5}$$

where $s_0 = \mathbf{0}$. Although the memory module and output module share the same inputs, the dynamics are different. First, the output module focuses on the processing of external input x and aims to solve the gradient problem across recurrent layers, whereas the memory module focuses on the processing of memory state s_{t-1} and amis to maintain long short-term memory. Second, it is notable that y_t is the output of the PR layer and feeds forward to the next layer in the RNN network (it no longer feeds back to the recurrent layer), whereas the output of the memory module is fed back by the recurrent connection. Compared with the FR layer, the parameters of the PR layer can easily be deployed for memories of previous events and standard deep nonlinear processing.

B. Implementation of PR Layer with Highway Connections

To learn the complex dependencies in sequence data, the recurrent layer should have three crucial abilities: maintaining long short-term memory, to learn long-term dependencies; deep transition mapping, to learn the complex space transformations in sequence data; and good gradient information flow across recurrent layers, to allow the training of stacked RNNs. Effective implementation of the PR layer, aiming to explore the aforementioned abilities, is presented by taking inspiration from RHNs.

As illustrated in Fig.3, we use two highway networks to implement the memory state transition $\mathcal{R} : s_{t-1}, x_t \rightarrow s_t$ and forward nonlinear mapping $\mathcal{F} : s_{t-1}, x_t \rightarrow y_t$ of the PL layer in Fig.2. Therefore, both the output module and memory module contain a highway network, which consists of L highway blocks in Fig.1b. In the output module, the external input x_t is placed on the highway connection while the memory state s_{t-1} is placed on the highway connection in the memory module. Furthermore, in the intermediate highway block l, the input information and memory information are exchanged between the highway blocks in the output module and memory module.

Generally, the dimensionality of an output module and memory module in the PR layer are m and n, i.e., $y_t \in \mathbb{R}^n$ and $s_t \in \mathbb{R}^m$. Given an input sequence $\mathbf{x} = \{x_t \in \mathbb{R}^n | t = 1, 2, \cdots, T\}$, the output of the PR layer can be computed step by step and layer by layer. To start the computation, the initial memory state $s_0^L \in \mathbb{R}^m$ is initialized by $s_0^L = \mathbf{0}$. At each time step t, let $y_t^0 = x_t$ and $s_t^0 = s_{t-1}^L$ for convenience.

For $l = 1 \rightarrow L$, the computation of each sublayer in the PR layer in Fig.2 can be formulated as:

$$\begin{pmatrix} y_t^l \\ s_t^l \end{pmatrix} = \begin{pmatrix} y_t^{l-1} \\ s_t^{l-1} \end{pmatrix} \cdot \left[\mathbf{1} - \begin{pmatrix} r_t^l \\ z_t^l \end{pmatrix} \right] + \begin{pmatrix} h_t^l \\ g_t^l \end{pmatrix} \cdot \begin{pmatrix} r_t^l \\ z_t^l \end{pmatrix}, \tag{6}$$

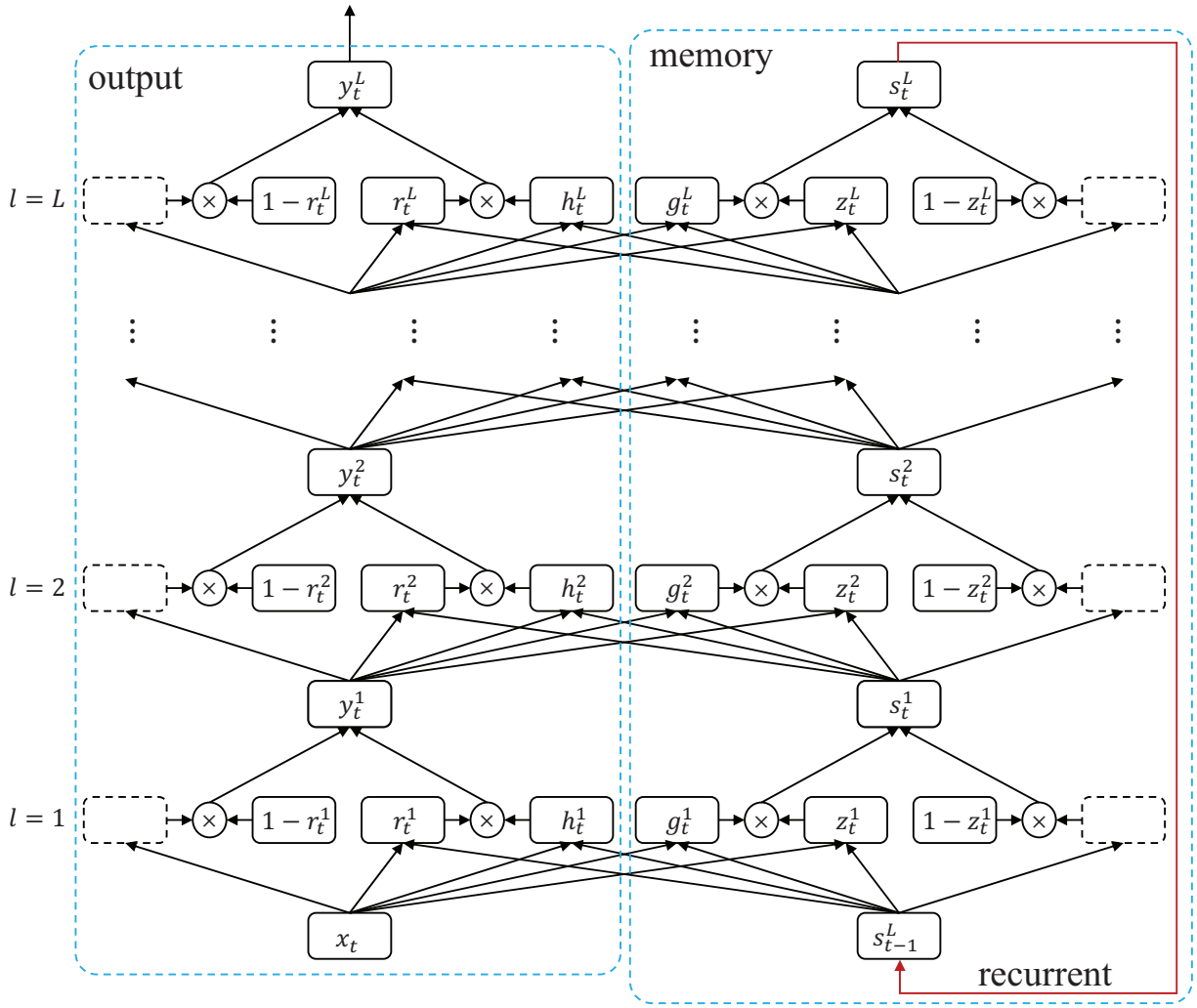


Fig. 3. The architecture of the proposed PR layer with highway connections, and the information flow in the PR layer . The output and memory modules are implemented as highway network with L highway blocks. '$\times$' denotes elementwise multiplication. The dotted square represents the identity operation and its connection has a fixed weight 1. r_t^l, g_t^l, z_t^l, and h_t^l are the output of nonlinear transforms with trainable weight connections. y_t^l and s_t^l are summations of the connected '$\times$' nodes, respectively.

where '$\cdot$' denotes the elementwise multiplication and r_t^l, h_t^l, z_t^l, and g_t^l are computed by

$$\begin{pmatrix} r_t^l \\ h_t^l \\ z_t^l \\ g_t^l \end{pmatrix} k = \begin{pmatrix} \sigma \\ \tanh \\ \sigma \\ \tanh \end{pmatrix} \begin{pmatrix} W_r^l \\ W_h^l \\ W_z^l \\ W_g^l \end{pmatrix} \begin{pmatrix} y_t^{l-1} \\ s_t^{l-1} \end{pmatrix}. \quad (7)$$

Finally, y_t^L serves as the output of the PR layer while s_t^L serves as the memory state and feeds back to the PR layer at the next time step:

$$\begin{cases} y_t = y_t^L \\ s_t = s_t^L \end{cases} \quad (8)$$

As illustrated in Eq. (6), the dimensionality of x_t, y_t^l, r_t^l, and h_t^l for all sublayer l should be the same. A transform layer can be inserted to change the dimensionality of the x_t to satisfy this requirement when necessary.

C. Deep PR-Net with PR Layers

The proposed PR layer in Fig.3 could be easily stacked to construct the deep PR-Net, which is analogous to stacking multiple hidden layers in the FNN [15]. Because of the proposed PR layer with highway connections, the PR-Net can overcome the problems of the fully recurrent layer and improve the performance of RNNs for complex sequence learning.

Compared with the LSTM network and RHN, the proposed PR-Net has comparable potential for the learning of long short-term memory.

From Eq. (6) and Eq. (8), we obtain:

$$s_t = s_{t-1} + \sum_{l=1}^{L} z_t^l \cdot (g_t^l - s_t^l). \quad (9)$$

Similar to Eq. (3) for a highway block, we have:

$$\frac{\partial s_t}{\partial s_{t-1}} = \begin{cases} \mathbf{I}, & z^l = \mathbf{0},\ \forall l \in [1, L] \\ \frac{\partial \sum_{l=1}^{L} z_t^l \cdot (g_t^l - s_t^l)}{\partial s_{t-1}}, & z^l = \mathbf{1},\ \forall l \in [1, L]. \end{cases} \quad (10)$$

This implies that the proposed PR layer has good gradient information flow through time, to allow the learning of long short-term memory. Furthermore, the memory module of the PR layer is the same as an RHN layer; the good gradient properties of such networks have also been analyzed by using GCT in [15].

The training of deep PR-Net is easier than that of stacked LSTM and RHN networks. From Eq. (6), we obtain:

$$y_t = x_t + \sum_{l=1}^{L} r_t^l \cdot (h_t^l - y_t^l). \tag{11}$$

Similarly, the following equation holds:

$$\frac{\partial y_t}{\partial x_t} = \begin{cases} \mathbf{I}, & r^l = \mathbf{0},\ \forall l \in [1, L] \\ \frac{\partial \sum_{l=1}^{L} r_t^l \cdot (h_t^l - y_t^l)}{\partial s_{t-1}}, & r^l = \mathbf{1},\ \forall l \in [1, L]. \end{cases} \tag{12}$$

The proposed PR layer in PR-Net can smoothly adapt its behavior from a complex nonlinear network to a layer that passes its input unimpeded; this ability is analogous to that of the highway block in highway networks [19]. This implies that the gradient information can be backpropagated through the recurrent layer easily. However, neither the LSTM nor the RHN network has the same character.

In the same manner as the RHN layer, the PR layer in PR-Net also contains deep transition mappings to learn the complex space transformations from a step t to the next step $t+1$, which is absent from the LSTM network. In addition, the PR-Net does not suffer from the problems of the fully recurrent network because it separates the memory and output explicitly.

The motivation of the proposed PR-Net is inspired by the AMU model [12]. However, they are two totally different models. On the one hand, the output neurons in the proposed PR-Net are feedforward neurons while they are recurrent neurons in the AMU. On the other hand, we implement the PR-Net with highway connections. Thus the computations of the output block and memory block in the PR-Net are different from those in the AMU. Compared to AMU, the advantages of the PR-Net lies in two aspects. One is the good gradient properties in the stacked deep RNN models as implies in the Eqt. (12). The other is the update strategy of memory. In PR-Net, the memory block contains deep transition mappings to learn the complex space transformations from a step t to the next step $t+1$.

IV. Experiments

To evaluate the proposed PR-Net, we compared it with the state-of-the-art recurrent models, such as LSTM, RHN, AMU, and IndRNN. The experiments were conducted on three benchmarks, i.e., pixel-by-pixel MNIST classification, word-level PTB language modeling, and polyphonic music modeling. For a fair comparison, all of the recurrent models shared the same setup settings and contained comparable numbers of units or trainable parameters for each task.

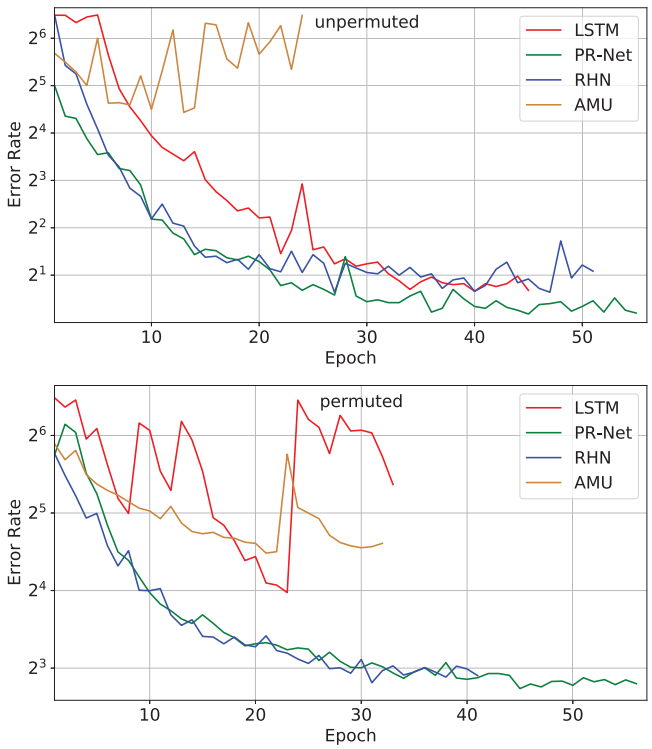


Fig. 4. The error rate curves of the recurrent models on the validation set for pixel-by-pixel MNIST classification.

A. Pixel by Pixel MNIST Classification

Pixel-by-pixel MNIST classification has been widely used to evaluate the ability of recurrent models in the learning of long-term dependencies. For this task, the input image $x_{img} \in \mathbb{R}^{28\times 28}$ of each data sample in the original MNIST dataset is represented by a sequence of its pixels. Generally, there are two settings for obtaining the pixel-by-pixel sequence of an image [8], [21], [22]. One is called the unpermuted setting, in which the image is serialized to a sequence $\mathbf{x} = \{x_t \in \mathbb{R}^1 | t = 1, 2, \cdots, 784\}$. The other is the permuted setting, in which each pixel sequence is rearranged to a fixed permutation. To perform early stop, we randomly selected $5,000$ samples from the original $60,000$ training samples as a validation dataset and the other $55,000$ samples were used as training dataset [22]. In addition, the test dataset contains $10,000$ samples.

For this task, LSTM, RHN, AMU, and PR-Net had three layers, an input layer with one neuron, a softmax output layer with 10 neurons, and a hidden recurrent layer. While the IndRNN has six RNN layers. The number of parameters are presented in Table II. The number of sublayers of the recurrent layer, in the RHN model and PR-Net model, was set to $L = 1$. Cross-entropy was used as the loss function. LSTM, RHN, and PR-Net used RMSProp optimization method while the AMU and IndRNN used the Adam optimization method. The early stop technique was performed with patience 10 and the weights with the lowest validation loss were used to evaluate the testing set.

The learning curves and a summary of the error rate results are presented in Table II and Fig.4, respectively. The result of IndRNN is from Ref. [23] since we failed to reproduce the

TABLE II
SUMMARY OF NETWORK SETTINGS AND ERROR RATES OF RECURRENT MODELS FOR PIXEL-LEVEL MNIST CLASSIFICATION. THE LOWEST VALUES OF EACH COLUMN ARE HIGHLIGHTED.

Models	Hidden Size	# params	Unpermuted		Permuted	
			Validation	Test	Validation	Test
LSTM	128	$\approx 679K$	1.86	1.56	15.72	15.22
RHN	128	$\approx 346K$	1.22	1.27	7.02	7.16
AMU	128	$\approx 675K$	21.64	21.64	22.36	21.22
IndRNN	128	$\approx 879K$	-	**1.18**	-	-
PR-Net	96, 32	$\approx 255K$	1.18	**1.16**	**6.94**	**6.70**

claimed performance. As Table II shows, IndRNN achieved the lowest error rate at the unpermuted settings by using large-scale parameters. The proposed PR-Net achieved comparable results with the fewest parameters.

As illustrated in Fig.4, the proposed PR-Net and RHN outperformed the LSTM and AMU models significantly, with more stable learning curves and lower error rates, particularly under the permuted settings. Furthermore, the proposed PR-Net outperformed RHN slightly, with a lower error rate. This demonstrates that the ability of PR-Net to learn long-term dependencies in sequential data is comparable to that of the LSTM and RHN networks.

B. Word Level Language Modeling on PTB

Peen Treebank (PTB) [8], [15] is a popular benchmark for evaluating the performance of RNN-based language modeling methods. There are $42,068$, $3,370$, and $3,761$ sentences in the training, validation, and test datasets respectively. The dictionary size is limited to 10000. The evaluation metric used is perplexity (PPL), which is defined as :

$$\mathbf{PPL} = 2^{-\frac{1}{T}\sum_{t=1}^{T}\log_2 y_t[d_t]}, \tag{13}$$

where y_t is the output of the RNN and d_t is the index of a word in the dictionary. A lower value of perplexity indicates a better result.

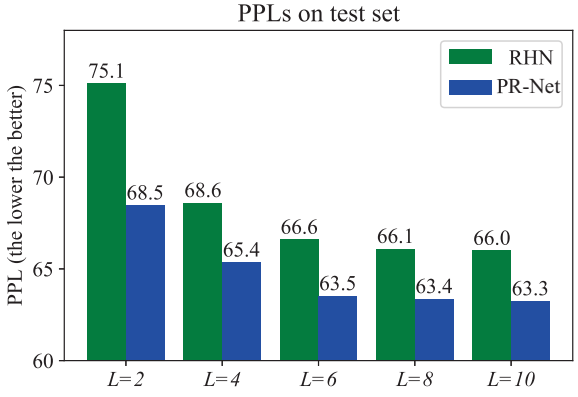


Fig. 5. Test set PPLs for PTB word-level language modeling using RHNs and PR-Net. L is the number of sublayers in the recurrent layer. As L increases, the number of parameters of RHNs increases from $20M$to $23M$, while the number of parameters of the proposed PR-Net is fixed to $25M$.

For this task, we took RHN [15] as the baseline, and our code was implemented by extending that of [15]. Therefore the running setups and hyperparameters were the same as those of RHN in [15]. Specifically, all of the networks contained only one recurrent layer.

To demonstrate the superior performance of the PR layer, we first compared the PR-Net with the RHN network. The results of the PR-Net and RHN network with different deep transition mapping networks are presented in Fig. 5. The performance of the PR-Net and RHN network improved as L was increased. It is interesting that the proposed PR-Net always outperformed the RHN with the same L. In addition, the PPL of the PR-Net with $L = 4$ was lower than that of the RHN with $L = 10$. This demonstrates that the PR-Net outperformed the RHN network, in terms of the learning of the complex space transform.

TABLE III
RECENT STATE-OF-THE-ART RESULTS FOR WORD-LEVEL LANGUAGE MODELING ON PTB. BEST PPL (THE LOWER THE BETTER) ON THE VALIDATION SET AND TEST ARE PRESENTED IN DESCENDING ORDER. "DROPOUT" AND "VARIATIONAL" INDICATE THE REGULARIZATION METHOD USED IN [24] AND [25], RESPECTIVELY. "WT" REPRESENTES THE TECHNIQUE OF WEIGHT-TYING PROPOSED IN [26].

Model	Size	Best Val.	Test
LSTM+dropout [24]	66M	82.2	78.4
Variational LSTM + WT [27]	66M	77.3	75.0
Variational RHN [15]	32M	71.2	68.9
NAS with base 8 [28]	32M	—	67.9
Variational RHN + WT [15]	23M	67.9	65.4
res-IndRNN [23]	—	—	65.3
NAS with base 8 + WT [28]	25M	—	64.0
Variational PR-Net + WT	**25M**	**65.7**	**63.3**
NAS with base 8 + WT [28]	54M	—	62.4

Furthermore, we compared the PR-Net with state-of-the-art models on this dataset. The PLLs of these models are presented in Table III. With comparable parameters, the proposed PR-Net outperformed most of the single models, except for the model using neural architecture search (NAS).

C. Polyphonic Music Modeling on Nottingham

Polyphonic music modeling is a class of sequence generation tasks. Following [14], given a training dataset with N polyphonic music sequences, the aim of RNN is to minimize the negative log-likelihood function:

$$L = -\frac{\sum_{n=1}^{N}\sum_{t=1}^{T_n}\log y_t^n}{\sum_{n=1}^{N} T_n}, \tag{14}$$

where y_t^n is the output of recurrent neural network at time step t for n-th sequence and T_n is the length of n-th sequence. For

TABLE IV
THE NEGATIVE LOG-LIKELIHOOD VALUES (THE LOWER THE BETTER) OF RECURRENT MODELS ON NOTTINGHAM DATASET, WHERE "VALID" DENOTES VALIDATION SET.

Models	$D = 1$		$D = 4$		$D = 6$		$D = 8$	
	valid	test	valid	test	valid	test	valid	test
LSTM	3.60	3.67	3.73	3.86	4.44	4.61	4.44	4.69
RHN	3.78	3.83	9.97	10.23	9.96	10.22	9.99	10.26
AMU	3.39	3.44	3.44	3.55	3.70	3.90	4.15	4.32
IndRNN	-	-	5.15	5.15	5.85	6.04	6.22	6.48
PR-Net	**3.33**	**3.36**	**3.26**	**3.27**	**3.33**	**3.45**	**3.37**	**3.38**

this task, the Nottingham dataset from [29] was used. The train, validation, and test dataset consisted of 694, 173, 170 sequences, respectively.

TABLE V
NUMBER OF PARAMETERS OF RECURRENT MODELS.

Models	$D = 1$	$D = 4$	$D = 6$	$D = 8$
LSTM	$\approx 114K$	$\approx 461K$	$\approx 692K$	$\approx 924K$
RHN	$\approx 121K$	$\approx 468K$	$\approx 700K$	$\approx 932K$
AMU	$\approx 175K$	$\approx 696K$	$\approx 1043K$	$\approx 1390K$
IndRNN	-	$\approx 68K$	$\approx 97K$	$\approx 127K$
PR-Net	$\approx 78K$	$\approx 317K$	$\approx 476K$	$\approx 636K$

For this task, we would like to evaluate the network performance with different network depth D (the number of stacked recurrent layers). A summary of network settings is presented in Table V. Notably, IndRNN should consist of multiple layers to obtain better performance [23]. Thus IndRNN with only one layer was not evaluated in this experiment. LSTM, RHN, AMU, and PR-Net used RMSProp optimization method while the IndRNN used the Adam optimization method. Early stop was performed with patience 10. Following [14], the negative log-likelihood was used as the performance metric.

The learning curves and the negative log-likelihood values of three recurrent models with different network depths are presented in Table IV and Fig.6, respectively. The proposed PR-Net achieved the lowest negative log-likelihood value for different numbers of stacked layers. As the number of stacked recurrent layers increased, the performance of the compared recurrent models decreased significantly, whereas PR-Net always achieved state-of-the-art performance. As illustrated by the loss curves in Fig. 6, the learning of PR-Net is more stable and lower than the other models, which demonstrates the effectiveness of the highway connections in the output block in the proposed PR-Net. Besides, the learning of LSTM seems to be more unstable as the increases of stacked layers. It reveals that the LSTM model is easily trapped into local minimal when stacked with many LSTM layers.

V. CONCLUSION

In this paper, a PR layer with highway connections was proposed, to address the learning problems in RNNs. In the proposed PR layer, the neurons are explicitly separated into an output module and a memory module. By deploying highway networks, the memory module can maintain long short-term memory, while the output module can pass the gradient information through recurrent layers easily, to allow the training of deep RNNs. Furthermore, the deep transition mappings in the PR layer enable the learning of complex space transforms contained in sequence data. The PR layer is used to construct a deep PR-Net, which consists of the proposed PR layer with highway connections. Experiments on three sequence learning benchmarks demonstrated the superior performance of the proposed PR-Net.

REFERENCES

[1] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," in *International conference on machine learning*, 2015, pp. 2048–2057.

[2] H. Lee, M. Jung, and J. Tani, "Recognition of visually perceived compositional human actions by multiple spatio-temporal scales recurrent neural networks," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 10, no. 4, pp. 1058–1069, 2017.

[3] K. Sasaki and T. Ogata, "Adaptive drawing behavior by visuomotor learning using recurrent neural networks," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 11, no. 1, pp. 119–128, 2018.

[4] B. Yang, J. Cao, N. Wang, and X. Liu, "Anomalous behaviors detection in moving crowds based on a weighted convolutional autoencoder-long short-term memory network," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 11, no. 4, pp. 473–482, 2018.

[5] H. Tang, B. Xiao, W. Li, and G. Wang, "Pixel convolutional neural network for multi-focus image fusion," *Information Sciences: An International Journal*, vol. 433-434, pp. 125–141, 2018.

[6] D. Zhang, D. Meng, and J. Han, "Co-saliency detection via a self-paced multiple-instance learning framework," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 5, pp. 865–878, 2017.

[7] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3156–3164.

[8] J. Wang, L. Zhang, Y. Chen, and Z. Yi, "A new delay connection for long short-term memory networks," *International journal of neural systems*, vol. 28, no. 06, p. 1750061, 2018.

[9] A. Graves, A.-r. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *2013 IEEE international conference on acoustics, speech and signal processing*. IEEE, 2013, pp. 6645–6649.

[10] A. Graves and N. Jaitly, "Towards end-to-end speech recognition with recurrent neural networks," in *International conference on machine learning*, 2014, pp. 1764–1772.

[11] Z. Yi, *Convergence analysis of recurrent neural networks*. Springer Science & Business Media, 2013, vol. 13.

[12] J. Wang, L. Zhang, Q. Guo, and Z. Yi, "Recurrent Neural Networks With Auxiliary Memory Units," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 5, pp. 1652–1661, may 2018. [Online]. Available: http://ieeexplore.ieee.org/document/7883962/

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *NIPS 2014 Workshop on Deep Learning, December 2014*, 2014.

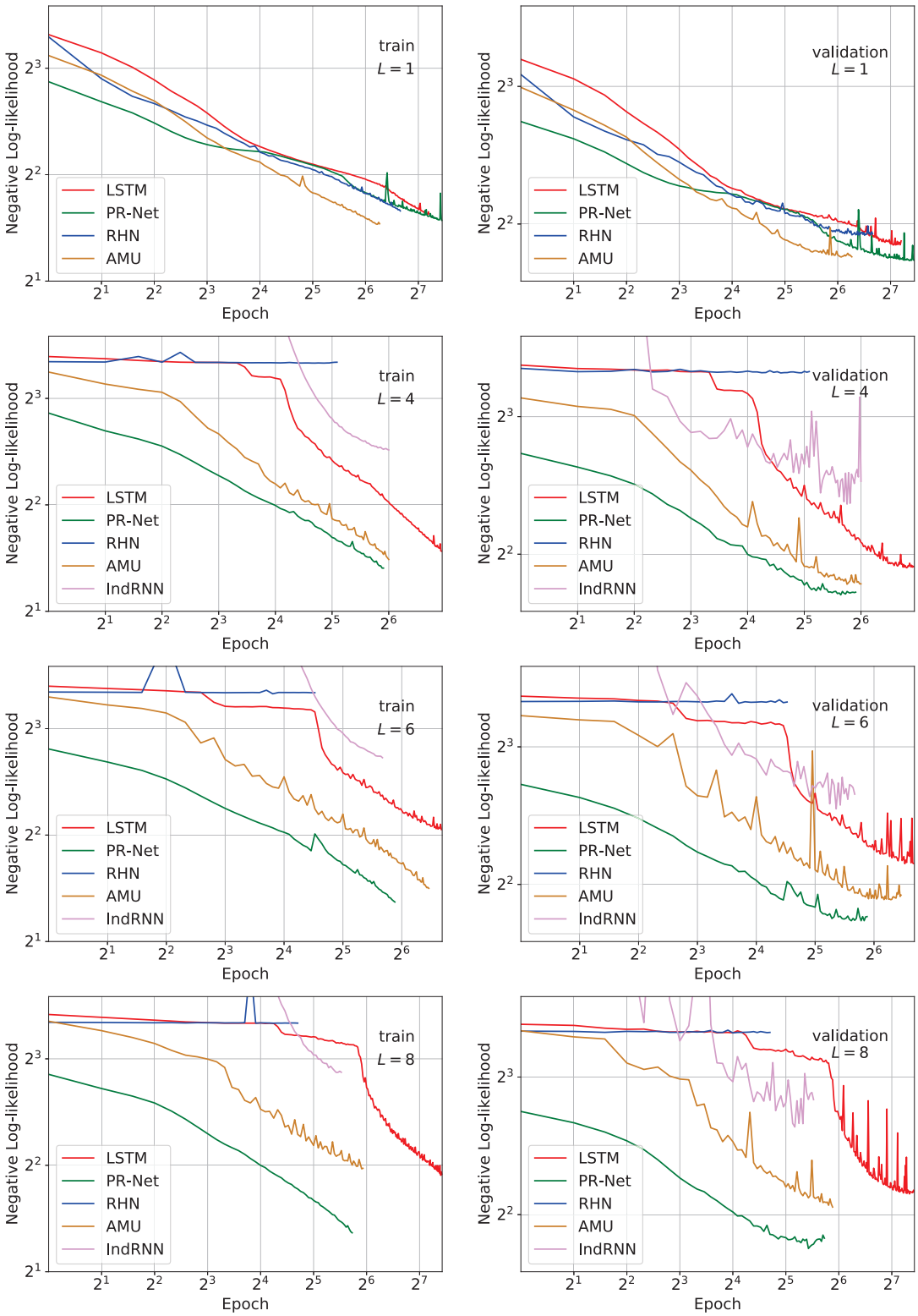


Fig. 6. The learning curves of the recurrent models on the Nottingham dataset.

[15] J. G. Zilly, R. K. Srivastava, J. Koutník, and J. Schmidhuber, "Recurrent highway networks," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, pp. 4189–4198. [Online]. Available: http://dl.acm.org/citation.cfm?id=3305890.3306114

[16] X. Li and X. Wu, "Constructing long short-term memory based deep recurrent neural networks for large vocabulary speech recognition," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, pp. 4520–4524.

[17] Y. Zhang, G. Chen, D. Yu, K. Yaco, S. Khudanpur, and J. Glass, "Highway long short-term memory rnns for distant speech recognition," in *2016 IEEE International Conference on Acoustics, Speech and Signal*

Processing (ICASSP). IEEE, 2016, pp. 5755–5759.
[18] A. Graves, "Adaptive computation time for recurrent neural networks," *CoRR*, vol. abs/1603.08983, 2016. [Online]. Available: http://arxiv.org/abs/1603.08983
[19] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training Very Deep Networks," in *Advances in Neural Information Processing Systems 28*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds. Curran Associates, Inc., 2015, pp. 2377–2385. [Online]. Available: http://papers.nips.cc/paper/5850-training-very-deep-networks.pdf
[20] J. L. Elman, "Finding structure in time," *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.
[21] Q. V. Le, N. Jaitly, and G. E. Hinton, "A simple way to initialize recurrent networks of rectified linear units," *arXiv preprint arXiv:1504.00941*, 2015.
[22] S. Wisdom, T. Powers, J. Hershey, J. Le Roux, and L. Atlas, "Full-capacity unitary recurrent neural networks," in *Advances in Neural Information Processing Systems*, 2016, pp. 4880–4888.
[23] S. Li, W. Li, C. Cook, C. Zhu, and Y. Gao, "Independently recurrent neural network (indrnn): Building a longer and deeper rnn," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 5457–5466.
[24] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," *arXiv preprint arXiv:1409.2329*, 2014.
[25] Y. Gal and Z. Ghahramani, "A theoretically grounded application of dropout in recurrent neural networks," in *Advances in Neural Information Processing Systems*, 2016, pp. 1019–1027.
[26] H. Inan, K. Khosravi, and R. Socher, "Tying word vectors and word classifiers: A loss framework for language modeling," *arXiv preprint arXiv:1611.01462*, 2016.
[27] O. Press and L. Wolf, "Using the output embedding to improve language models," in *European Chapter of the Association for Computational Linguistics*, ser. EACL, 2017.
[28] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*, 2017.
[29] N. Boulanger-Lewandowski, Y. Bengio, and P. Vincent, "Modeling temporal dependencies in high-dimensional sequences: application to polyphonic music generation and transcription," in *Proceedings of the 29th International Coference on International Conference on Machine Learning*. Omnipress, 2012, pp. 1881–1888.
[30] N. C. Dvornek, P. Ventola, K. A. Pelphrey, and J. S. Duncan, "Identifying autism from resting-state fmri using long short-term memory networks," in *International Workshop on Machine Learning in Medical Imaging*. Springer, 2017, pp. 362–370.
[31] J. Liu, G. Wang, P. Hu, L.-Y. Duan, and A. C. Kot, "Global context-aware attention lstm networks for 3d action recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1647–1656.
[32] Z. Xu, S. Li, and W. Deng, "Learning temporal features using lstm-cnn architecture for face anti-spoofing," in *2015 3rd IAPR Asian Conference on Pattern Recognition (ACPR)*. IEEE, 2015, pp. 141–145.
[33] A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint arXiv:1308.0850*, 2013.

Jianyong Wang (M'18) received his Ph.D. degree in computer science from the Sichuan University in 2018. He is currently a Post-Doctoral Research Fellow in the Machine Intelligence Lab at College of Computer Science, Sichuan University, Chengdu, China. His current research interests include Neural Networks and their applications in medical data analysis.

Haixian Zhang (M'14) received the Ph.D. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2011. She is currently an Associate Professor with the College of Computer Science, Sichuan University, Chengdu. Her current research interests include neural networks and big data.

Zhang Yi (F' 16) received the Ph.D. degree in mathematics from the Institute of Mathematics, The Chinese Academy of Science, Beijing, China, in 1994. Currently, he is a Professor at the College of Computer Science, Sichuan University, Chengdu, China. He is the co-author of three books: Convergence Analysis of Recurrent Neural Networks (Kluwer Academic Publisher, 2004), Neural Networks: Computational Models and Applications (Springer, 2007), and Subspace Learning of Neural Networks (CRC Press, 2010). He is a Fellow of IEEE. He is the Chair of IEEE Chengdu Section (2015 –). He was an Associate Editor of IEEE Transactions on Neural Networks and Learning Systems (2009 – 2012), and an Associate Editor of IEEE Transactions on Cybernetics (2014 –). He is also a Fellow of CAAI. His current research interests include Neural Networks and Big Data. He is the founding director of Machine Intelligence Laboratory. He is also the founder of IEEE Computational Intelligence Society, Chengdu Chapter.